Pedal Pro Rentals: Agentforce AI Assistant

I earned the Salesforce Agentforce Specialist certification in December 2025 (credential 7317949, verify it here or see the certificate PDF). This project is how I put it to work. I wanted to build something real instead of leaving the knowledge on a shelf.

It's a custom Agentforce agent for a fictional bike rental company. It checks which bikes are available, books a rental by creating an actual Salesforce record and updating the bike's status, and handles cancellations. There's also a real business rule built in: if someone tries to cancel a rental that starts within 2 hours, the agent won't do it. It explains the policy and creates a Case so a person can follow up.

Demo video



Click the image to play the video (it's also saved in this repo at `demo/DEMO.mp4`)

About 2.5 minutes, screen recording only, no narration. It shows the agent checking availability, booking a bike, the bike's status changing to Rented, and then cancelling a booking that's more than 2 hours away.

What it does

Feature	What actually happens
Check availability	The agent looks up bikes by type and location from live Salesforce data. It doesn't guess.
Book a rental	A Flow creates a `Rental__c` record and flips the bike's status to Rented in one go.
Cancel a booking	If the rental starts in more than 2 hours, it's cancelled. If it starts within 2 hours, the agent refuses and creates a Case for a human to review.

How it’s built

Data model

- `Bike__c` : model, type (Mountain, Road, Hybrid, Electric), location, status (Available, Rented, Maintenance), hourly rate
- `Rental__c` : bike, customer (Contact), start and end date/time, status, late fee

Five Flows, all Autolaunched, each one wrapped as an Agentforce action:

Flow	What it does
`Check_Bike_Availability`	Finds available bikes by type and location
`Book_Rental_Flow`	Creates the rental and marks the bike Rented
`Get_Booking_Name`	Looks up a booking and works out how many minutes are left until it starts
`Cancel_Booking`	Sets a rental’s status to Cancelled
`Escalate_To_Human`	Creates a Case for a person to review

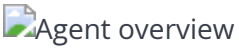
The agent

A router sends each customer to one of three subagents: Bike Availability, Bike Booking, or Booking Cancellation. Each has its own plain-English instructions and its own list of allowed actions.

The agent in Agentforce Studio:


Agents in Agentforce Studio

The router and its subagents in Agentforce Builder:


Agent overview

The live test panel I used to try it out, with the greeting a customer sees first:


Live test preview

Access

The agent runs as its own dedicated user, not as me. That user needed its own Permission Set (`Agentforce Bike Access`) before it could read or write any of the custom objects.

Walkthrough

1. Checking availability

The Flow looks for bikes that match the type and location and are still Available, then returns a found or none-found message.


Check availability flow

A debug run with `Electric` and `Madhapur` finds a match:


Debug input 
Debug result

2. Booking a rental

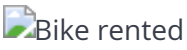
The booking Flow checks the bike is still Available, creates the rental, marks the bike Rented, and returns a confirmation ID. The dashed line is a Fault Path that I added while debugging (more on that below).


Book rental flow

The dates go in as plain text and get converted to real date/times inside the Flow:


Debug input 
Debug trace

The same bike before and after the booking:

Before: Available	After: Rented
 Bike available	 Bike rented

3. Cancelling, and the guardrail

The rule is: no AI-driven cancellations within 2 hours of the start time.

My first version told the agent to check a variable called `minutes_until_start`. The problem was that nothing in the project ever calculated it, so the agent just cancelled everything. The fix was to do the math inside a Flow, where it belongs. `Get_Booking_Name` has a formula that works out the minutes until the rental starts:

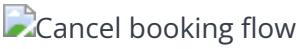
```
(Start_Date_Time__c - NOW()) * 1440
```

Subtracting two date/times gives a number of days, and multiplying by 1440 turns that into minutes. The agent gets back a real number called `minutesUntilStartOutput` and branches on it:

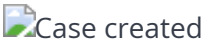
- More than 120: cancel the booking
- 120 or less: create a Case and tell the customer a team member will follow up

 Get booking name flow

The cancel path, which only runs when the booking is more than 2 hours out:

 Cancel booking flow

The escalation path, which creates the Case instead of cancelling:

 Escalate input  Case created

I tested both paths through the agent. A rental about 88 minutes out was refused and a real Case was created. A rental about 814 minutes out was cancelled normally.

To test the guardrail I needed a booking that starts soon, so I made test bookings by running the booking Flow directly in Debug:

 Guardrail test booking input  Decision default outcome

The second image is one of those runs landing on the “bike not available” branch. The Flow handles that by returning a message instead of booking, which is exactly what it should do when the bike has already been taken.

The subagents

▶ Instructions for each subagent (click to expand)

The test data

 Bikes  Rentals

Problems I ran into

Most of my time went into debugging, not building. These are the ones I learned the most from.

1. It worked in Debug but failed when the agent called it.
This kept happening. Debug runs as me, the admin, with every permission. The agent

runs as its own user. The first cause was that this user had no access to my custom objects at all, which I fixed with a Permission Set.

2. Date/Time inputs failed silently.
When the agent called the booking Flow, the date fields showed up as a generic "Object" type and the Flow failed without saying why. I changed the Flow to take the dates as plain text and convert them inside the Flow with `DATETIMEVALUE()`.

3. The agent said a booking worked when it hadn't.
No Rental record existed and the bike's status never changed. I rewrote the instructions so the agent has to see `success = true` from the action before it confirms anything, and so it says honestly when the booking didn't go through.

4. A generic error was hiding the real one.
The booking kept failing with the same canned message. I added a Fault Path to the Flow and pointed it at `{!$Flow.FaultMessage}` so I could read Salesforce's actual error. It turned out to be an "insufficient access rights" error on one specific Contact record.

5. Object permission isn't record access.
That Contact error was a record-level problem, not an object-level one. The agent's user had full permission on the object but still couldn't touch a record it didn't own and wasn't shared on. I fixed it by changing the record owner. This was the sneakiest one, because everything looked correct.

6. The agent changed my picklist values.
A customer said "electric bike" and the agent sent `e-bike` to the Flow. The Flow does an exact match, so it found nothing. I fixed it by listing the exact allowed values in the instructions.

Known limitations

- The agent asks the customer for a raw Contact ID, which no real customer would know. A real version needs a lookup by email or phone.
- Availability returns a plain "bikes available" or "none available" message, not a list of bikes. An earlier version tried to build a list with a Loop, and it kept failing when called live, so I simplified it.
- My org's `NOW()` was offset from my local time, so I tested the guardrail by creating a record, running Debug, reading the real number, and adjusting the start time until it landed where I needed.
- Records the agent works with have to be accessible to its dedicated user.

What I'd add next

- A "find customer by email or phone" action, so nobody has to type an ID
- A proper bike list in the availability results, with names, rates, and IDs
- Late fee calculation when a bike is returned
- Fixing the org time zone so test data doesn't need adjusting

What's in this repo

```
README.md
demo/
    DEMO.mp4
docs/
    Pedal_Pro_Rentals_Agentforce_Project_Master_Guide.pdf
    Pedal_Pro_Rentals_Agentforce_Study_Guide.pdf
    Agentforce_Specialist_Certificate.pdf
screenshots/
```

The two guides in `docs/` are my full build notes: every object and field, every Flow, the final agent instructions, and a log of every bug and fix. I wrote them mainly for my own reference.

About me

Srinivas Sayinni. Salesforce Agentforce Specialist certified.

LinkedIn